Real-world (compare your loop to the library's) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- Hint 1 — `create_agent` is your loop, pre-built
- Hint 2 — The plan, and almost the whole thing

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 1 — `create_agent` is your loop, pre-built

Basic Version

`create_agent(model, tools=ALL_TOOLS)` (from `langchain.agents`) builds the same think → act → observe loop you just wrote. You call it with `agent.invoke({"messages": [{"role": "user", "content": ...}]})`, and the result's `"messages"` list holds the whole run — the last message is the answer.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

To compare fairly, run the **same prompts** through both loops and count tool calls:

- yours: `len(run_agent(prompt).steps)`;
- the library's: count the messages whose `.type` is `"tool"` — one per tool result.

And compare how each stops a run that never ends: your `MaxIterationsExceeded`, and the library's `recursion_limit` setting — `agent.invoke(inputs, {"recursion_limit": 6})` — which raises `GraphRecursionError` (from `langgraph.errors`). It counts graph steps, about 2 per turn.

Difference between Basic and Intermediate: Basic runs the library once. Intermediate compares both engines on the same prompts, and on a run that never ends.

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
agent = create_agent(model, tools=ALL_TOOLS)
result = agent.invoke({"messages": [the question]})
print the last message's content
```

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

```
for each of 3 prompts:
    mine = run_agent(prompt)            # from react_loop_practice.py
    library = agent.invoke(...)
    print both tool-call counts

looping_agent = create_agent(model, tools=[always_ask_again])
try: looping_agent.invoke(..., {"recursion_limit": 6})
except GraphRecursionError: print it
try: run_agent(two-step task, max_iterations=1)
except MaxIterationsExceeded as e: print len(e.steps)
```

Write it, then compare against the Solution.

Difference between Basic and Intermediate: Basic shows the library works. Intermediate proves it does the same thing as your loop — and stops the same way.

Back to the exercise · Hint 1 · Hint 2 · Solution

Full solution: Show me the solution